

Vibe Coding 从0到1开发教程

基于 NovaLeap 全栈项目的 B 站录课讲稿

这份 PDF 适合怎么用

它不是一份生硬的技术说明，而是一份可以拿来录视频的教程脚本。你可以按章节录制：先讲方法论，再讲 NovaLeap 项目，再讲每个阶段怎么和 AI 协作，最后用完整口述稿收尾。

项目	内容
项目名称	NovaLeap
项目定位	学习、内容沉淀、AI 辅助和社区互动的全栈平台
项目组成	用户端 nova-frontend、管理端 nova-admin、后端 nova-backend
技术栈	Vue 3、Vite、Pinia、Spring Boot、MyBatis-Plus、MySQL、Redis、Spring AI、Docker Compose
录课主题	如何用 Vibe Coding 从0到1完成一个真实全栈项目

目录和录制路线

章节	视频里怎么讲	建议时长
1. 先讲 Vibe Coding 是什么	强调它不是让 AI 替你写，而是你控方向，AI 提效率。	3 分钟
2. 展示 NovaLeap 项目	讲清楚用户端、管理端、后端三部分。	4 分钟
3. 前期筹备	立项、功能清单、技术选型、架构、Agent 宪法。	8 分钟
4. 开发迭代	实施真源文档、分阶段开发、每阶段验收。	10 分钟
5. 项目落地	联调、权限、AI 流式返回、Docker 部署。	8 分钟
6. 面试口述	把项目经历整理成能自然说出来的一段话。	5 分钟

录制主线

整期视频不要只讲 AI 写代码，要讲清楚一个核心：我先把地基、规则和验收标准立好，再让 AI 按阶段执行。这样 Vibe Coding 才是可控的工程流程。

1. NovaLeap 项目概览

录视频时可以先用一两分钟介绍项目，让观众知道这不是一个空泛案例，而是一个真实的全栈项目。

一句话介绍

NovaLeap 是一个集题库练习、学习笔记、AI 简历分析、AI 陪练、社区互动和后台管理于一体的学习平台。

项目分成三个部分：

部分	作用	核心功能
用户端 nova-frontend	给普通用户使用	注册登录、题库、笔记、AI 简历分析、AI 陪练、愿望墙、排行榜、个人中心
管理端 nova-admin	给管理员和运营使用	管理员登录、数据总览、用户管理、题目管理、笔记审核、愿望墙审核、访客记录、系统监控
后端 nova-backend	统一提供业务接口和基础能力	JWT 鉴权、MySQL 持久化、Redis 缓存和排行榜、AI 接入、邮件验证码、Docker 部署

讲解口径：

- 我做的不是单纯页面 Demo，而是用户端、管理端和后端都完整的全栈项目。
- 用户端解决学习和 AI 辅助，管理端解决内容运营，后端负责数据、权限和部署。
- 这个项目很适合用来讲 Vibe Coding，因为它既有产品拆解，也有真实联调和部署。

2. Vibe Coding 完整开发流程

我的流程可以概括成一句话：先立地基，再定规矩，再分阶段执行，最后验收和部署。

阶段	核心动作	在 NovaLeap 中的体现
前期筹备	立项、功能清单、技术选型、架构搭建、Agent 宪法	确定学习平台方向，拆成用户端、管理端、后端，固定 Vue + Spring Boot 技术栈
开发迭代	实施真源文档、分阶段推进、阶段验收	按登录、题库、笔记、AI、愿望墙、后台管理逐步完成
联调部署	权限联调、数据验证、AI 接入、Docker 部署、文档整理	统一 /api、JWT 鉴权、MySQL、Redis、Docker Compose 一键启动

2.1 前期筹备：动手写代码前

第一步是立项。不要一开始就问 AI 帮我写一个项目，而是先说明你要解决什么问题。

- 1 明确项目核心价值：NovaLeap 的价值是帮助用户学习、沉淀内容，并通过 AI 获得辅助反馈。
- 2 拆功能清单：把用户端、管理端、后端分别拆开，避免需求混在一起。
- 3 确定技术选型：前端 Vue 3，后端 Spring Boot，数据库 MySQL，缓存 Redis，AI 接 OpenAI 兼容接口。
- 4 搭最小可运行架构：先让用户端、管理端、后端可以独立启动，再逐步加功能。
- 5 制定 Agent 宪法：告诉 AI 不能乱改技术栈，不能随便重构，只能围绕当前阶段工作。

2.2 开发迭代：动手写代码后

- 1 先写实施真源文档：当前阶段做什么、不做什么、验收标准是什么。
- 2 让 AI 对照文档执行：不要让 AI 凭感觉扩展功能。
- 3 每个阶段完成后先验收：页面能不能打开，接口能不能通，数据能不能写入，权限是否正确。
- 4 把报错和现象反馈给 AI：让 AI 根据真实错误修复，而不是空想式改代码。
- 5 验收通过后再进入下一阶段：防止问题堆到最后一起爆发。

2.3 关键原则

原则	解释	你在视频里可以怎么说
地基红线	项目方向、技术栈、架构尽量不要中途摇摆。	Vibe Coding 最怕地基来回变，地基一变，AI 就容易越写越散。
防漂移	用实施真源文档和明确指令约束 AI。	我不会让 AI 随便发挥，而是让它只做当前阶段的任务。
验收优先	每个子阶段完成后先验证，再继续。	AI 写完不代表完成，我验收通过才算完成。
人为把关	AI 提效率，人控制方向和质量。	我不是复制 AI 代码，而是做产品负责人和技术把关人。

3. NovaLeap 分阶段开发路线

这一页适合放在视频中间，告诉观众一个真实项目不是一次性生成，而是按阶段推进。

阶段	模块	验收重点
阶段 1	登录注册和基础权限	完成普通用户登录、游客登录、管理员登录、JWT 鉴权，为后续模块打基础。
阶段 2	题库模块	完成题目列表、分类筛选、随机抽题、题目详情和答案查看。
阶段 3	笔记模块	完成笔记列表、我的笔记、详情、创建、编辑、点赞和评论。
阶段 4	AI 模块	完成 AI 简历分析、AI 陪练聊天、题目讲解，并处理流式返回。
阶段 5	社区和成长模块	完成愿望墙、排行榜、个人中心、连续签到等用户体验功能。
阶段 6	管理端	完成用户管理、题目管理、题目分类、笔记审核、愿望墙审核和系统监控。
阶段 7	联调和部署	统一接口、修权限、验证数据库和 Redis，最后用 Docker Compose 编排整体服务。

每个阶段的标准流程：

- 1 开工前：告诉 AI 当前阶段目标、任务边界和禁止事项。
- 2 开发中：让 AI 先读已有代码，再按当前项目风格实现。
- 3 完成后：要求 AI 总结修改内容、影响范围和验证方法。
- 4 人工验收：我自己运行项目，检查页面、接口、数据和权限。
- 5 通过后：记录阶段完成情况，再进入下一阶段。

4. Agent 宪法：给 AI 的项目规矩

这部分可以作为教程里的重点，因为它能体现你不是盲目让 AI 写代码，而是在管理 AI。

核心思想

Agent 宪法就是给 AI 的开发边界。它的作用是让 AI 每次开工前先理解项目，不乱改、不跑偏、不破坏已有结构。

可以这样规定 AI：

- 开工前先阅读项目结构、README、接口文档和相关模块代码。
- 保持现有技术栈，不要主动更换框架或引入无关依赖。
- 每次只围绕当前实施文档工作，不扩展无关功能。
- 接口命名、返回格式、目录结构要和已有代码保持一致。
- 不要随意大范围重构，除非当前任务明确需要。
- 涉及数据、权限和删除操作时必须谨慎说明。
- 完成后输出修改清单、验证结果和剩余风险。

录课时可以这样讲：

口述示例

我不会让 AI 每次自由发挥。我会先告诉它项目规则，比如不能随便换技术栈，不能乱重构，必须先看已有代码，每次只做当前阶段的任务。这样 AI 就像一个有规范的协作开发者，而不是一个想到哪写到哪的工具。

5. 实施真源文档模板

实施真源文档是防止 AI 漂移的核心。它告诉 AI 当前阶段唯一可信的任务来源是什么。

文档项	写什么	NovaLeap 示例
阶段目标	本阶段最终要完成什么	完成题库模块，支持题目列表、分类筛选、随机抽题和答案查看
任务边界	明确哪些事情不做	本阶段不改登录、不做笔记、不做 AI 模块
接口范围	需要哪些后端接口	/api/questions、/api/questions/categories、 /api/questions/random、/api/questions/{id}/answer
前端范围	需要哪些页面和状态	题库页、题目详情、分类筛选、加载状态、错误提示
验收标准	怎样才算完成	页面可访问，筛选生效，随机抽题可用，接口返回格式统一
验证方式	完成后怎么检查	本地运行前后端，手动测试主要流程，查看控制台和接口返回

给 AI 的提示词模板：

Prompt 模板

请先阅读当前项目结构和相关代码。现在只执行【题库模块】阶段，目标是完成题目列表、分类筛选、随机抽题和答案查看。不要修改登录、笔记、AI、后台管理等无关模块。完成后请说明修改了哪些文件、如何验证、还有哪些风险。

6. B 站录课分镜脚本

下面是你录视频时可以直接照着讲的结构。每一段都保持自然语言，不需要说得太学术。

片段	画面建议	讲解重点
开场	展示标题页或项目首页	今天讲我如何用 Vibe Coding 从0到1完成 NovaLeap 全栈项目。
项目展示	切换到用户端页面	先看项目结果：题库、笔记、AI 简历分析、AI 陪练、愿望墙和排行榜。
后台展示	切换到管理端	后台负责用户、题目、笔记审核、愿望墙审核和系统数据。
方法论	展示流程图或 PDF 第2章	Vibe Coding 不是让 AI 替我写，而是先立地基，再控执行。
前期筹备	展示功能清单	立项、拆功能、定技术栈、搭架构、写 Agent 宪法。
开发迭代	展示阶段表	按登录、题库、笔记、AI、后台管理逐步推进。
防漂移	展示实施真源文档模板	每阶段都写清楚目标、边界和验收标准。
联调部署	展示 docker-compose 或项目结构	真实项目要解决接口、权限、数据库、Redis 和部署。
面试总结	回到 PDF 口述稿	最后把这个项目讲成一段完整的面试回答。

7. 完整面试口述稿

这一段可以直接用于面试，也可以作为视频最后的总结。

我这个项目叫 NovaLeap，是一个面向学习、内容沉淀和 AI 辅助的全栈项目。它主要分成三个部分：用户端、管理端和后端服务。

用户端主要提供注册登录、题库练习、随机抽题、学习笔记、点赞评论、AI 简历分析、AI 陪练、愿望墙、排行榜和个人中心。管理端主要给管理员使用，可以做用户管理、题目管理、题目分类管理、笔记审核、愿望墙审核、访客记录和系统监控。后端负责统一接口、登录鉴权、数据库、缓存、AI 接入和部署。

我做这个项目的时候，用的是 Vibe Coding 的方式，但我不是简单地让 AI 一次性生成整个项目。我是先把项目地基搭好，再让 AI 分阶段参与开发。

一开始我先做立项。我想做的不是一个单纯的刷题网站，而是一个学习平台。用户不仅可以刷题，还可以写笔记、做简历分析、和 AI 练习对话，也可以通过愿望墙和排行榜形成互动。所以我先把项目定位成一个集题库、笔记、AI 辅助和社区互动于一体的学习平台。

确定方向后，我先拆功能清单。我把项目分成用户端、管理端和后端三块。用户端负责普通用户的学习体验，管理端负责运营和内容审核，后端负责数据、权限和 AI 能力。这样拆完以后，每个模块边界都比较清楚，AI 也不会乱写。

然后我做技术选型。前端用户端和管理端都用了 Vue 3、Vite、Pinia、Vue Router 和 Tailwind CSS。后端用了 Java 17、Spring Boot、Spring Security、MyBatis-Plus。数据库用 MySQL，缓存和排行榜用 Redis，AI 部分通过 Spring AI 接入 OpenAI 兼容接口，最后用 Docker Compose 做整体部署。

这里我有一个原则，就是技术栈确定后不轻易改。因为 Vibe Coding 很怕地基来回变，如果中途一直换技术栈，AI 生成的代码也会越来越乱，所以我会先把架构确定下来。

接下来我搭了最小可运行的项目架子。整个项目分成 nova-frontend、nova-admin 和 nova-backend。前台、后台和后端分开，但都走同一个后端接口体系。接口统一以 /api 开头，登录鉴权统一用 JWT，数据统一存在 MySQL，缓存和排行榜用 Redis。

在正式开发前，我还会给 AI 制定项目规则，也就是类似 Agent 宪法。比如开工前要先看已有代码和项目结构，不能随便换技术栈，不能随便大范围重构，每次只做当前阶段的任务，完成后要说明修改内容和验证方式。这样做主要是为了防止 AI 漂移。

开发阶段，我会把每一个大阶段拆成实施文档。比如做题库模块时，我会明确告诉 AI：这一阶段只做题库，不要碰笔记和 AI；要完成题目列表、分类筛选、随机抽题、查看答案；完成后前端能正常展示，后端接口能正常返回。这个实施文档就是 AI 执行的唯一依据。

然后我按阶段推进。第一阶段先做登录注册和权限，因为后面很多功能都依赖用户身份。第二阶段做题库，包括题目列表、分类、随机抽题和答案查看。第三阶段做笔记模块，包括创建、编辑、点赞、评论。第四阶段做 AI 模块，包括简历分析和 AI 陪练，这里我用了流式返回，让用户能看到 AI 一边生成一边展示。后面再做愿望墙、排行榜、个人中心。用户端跑通以后，再做后台管理端。

每个阶段我都不是让 AI 写完就直接进入下一个阶段，而是先验收。我会本地运行前端和后端，看页面能不能打开，接口能不能通，数据有没有写进数据库，权限有没有生效。如果有报错，我会把报错信息、相关代码和我期望的结果发给 AI，让它帮我定位。AI 给出修改后，我再自己检查和运行。

后期我主要做前后端联调和部署整理。比如检查 token 是否正确传递，普通用户和管理员权限是否区分，AI 接口是否能流式返回，MySQL 和 Redis 是否能正常连接。最后我整理了 Docker Compose，把前端、管理端、后端、MySQL、Redis 放到一套服务里，同时补了环境变量示例、API 文档和部署说明。

所以我总结我的 Vibe Coding 流程，就是先立项，再拆功能，再定技术栈和架构，然后制定 AI 规则，接着写实施真源文档，让 AI 按阶段开发，每个阶段完成后验收，最后做联调、部署和文档整理。

我觉得 Vibe Coding 的关键不是让 AI 替我写完整项目，而是我自己要能控制方向、拆清任务、约束边界、判断结果。AI 在这个过程中提高了开发效率，帮我生成基础代码、排查错误、补接口和整理文档；但项目的整体设计、模块边界、验收标准和最终整合，还是由我来把控。NovaLeap 就是我用这个流程，从一个学习平台的想法，一步步做成用户端、管理端、后端、AI 能力和 Docker 部署都完整的一套全栈项目。

8.1 分钟精简版

如果面试官只是简单问：你怎么用 Vibe Coding 做这个项目，可以用下面这版。

我不是直接让 AI 一次性生成项目，而是先把 NovaLeap 的定位、功能清单、技术栈和架构确定下来。这个项目定位是学习平台，包含题库、笔记、AI 简历分析、AI 陪练、愿望墙、排行榜和后台管理。

然后我把项目拆成用户端、管理端和后端三部分。前端用 Vue 3，后端用 Spring Boot，数据库用 MySQL，缓存用 Redis，AI 接入 OpenAI 兼容接口，部署用 Docker Compose。

开发时我会先写清楚每个阶段的实施文档，比如当前阶段只做登录，或者只做题库，明确任务边界和验收标准。然后让 AI 按这个文档生成代码，我再本地运行、联调、看报错、修问题。每个阶段验收通过后，再进入下一个阶段。

所以我的 Vibe Coding 流程就是：先立地基，再定规则，再分阶段执行，最后验收和部署。AI 主要帮我提高效率，但项目方向、模块拆分、质量验收和最终整合都是我自己控制的。

视频结尾金句：

总结

Vibe Coding 不是把方向交给 AI，而是我把方向、边界和验收标准定清楚，让 AI 在可控范围内把开发效率拉高。

9. 附录：可直接使用的提示词

场景	提示词
项目立项 Prompt	我要做一个学习类全栈项目，核心功能包括题库、笔记、AI 简历分析、AI 陪练、愿望墙、排行榜和后台管理。请帮我从用户角色、核心场景、功能模块和开发阶段四个角度拆解，不要直接写代码。
技术选型 Prompt	基于这个项目，我准备用 Vue 3 做前端，Spring Boot 做后端，MySQL 做数据库，Redis 做缓存，Docker Compose 做部署。请帮我判断这个技术栈是否适合，并列出具体的目录建议和模块边界。
阶段开发 Prompt	请先阅读项目结构和相关代码。当前只开发【某某模块】，不要修改无关模块。请按照现有代码风格完成实现，并在最后说明修改文件、验证方式和风险点。
报错修复 Prompt	下面是我本地运行时的报错和相关代码。请先判断最可能原因，再给出最小修改方案。不要重构无关代码，不要更换技术栈。
阶段验收 Prompt	请根据本阶段目标、任务边界和验收标准，检查当前实现是否完成。请输出完成项、未完成项、潜在问题和验证步骤。

附录：项目结构讲解

目录	说明
nova-frontend	普通用户端，承载题库、笔记、AI、愿望墙、排行榜等页面。
nova-admin	后台管理端，承载管理员登录、内容审核、用户和题目管理。
nova-backend	后端服务，提供 API、鉴权、数据库、缓存、AI 接入等能力。
docker-compose.yml	统一编排 MySQL、Redis、后端、用户端和管理端。
API_SPECIFICATION.md	接口说明文档，用于前后端联调和阶段验收。